State Management with React Context API

React JS

What is Context API?

Introduction

The **Context API** in React.js is a feature that enables **the sharing of data across components** without the need for **manual prop passing at every level of the component tree**. It provides a mechanism to create a "**global**" state or data that can be accessed by **any component** within a specified part of the application, regardless of its depth in the component hierarchy.

useContext() hook

The **useContext** hook in React allows functional components to easily access and consume values from a React Context. It's a powerful tool or hook for managing **global state** and **sharing data between components** without needing to manually pass props down through every level of the component tree, a problem commonly known as "prop drilling".

Advantages of using useContext hook.

- **Avoids Prop Drilling:** useContext helps prevent "prop drilling," where data is passed through multiple layers of components that don't need it. Components can access context values directly.
- **Simplifies State Management:** It offers a streamlined approach for managing global state without requiring complex external libraries for simpler applications.
- **Centralized State Management:** Context API provides a way to manage global application state in a centralized location.
- **Improved Code Readability:** Centralizing shared state makes components clearer and the codebase easier to maintain.

How Context API works?

Steps to create and use react context.

- **Create Context:**
  Use React.createContext() to create a context object for shared data.

- **Provide Context:**
  Wrap components needing access with a <MyContext.Provider>, passing the data via the value prop.

- **Consume Context:**
  Use the useContext hook in functional components within the Provider's tree, passing the context object to access the value from the nearest Provider.

Create Context

Use **createContext**() to create a context object for shared data.

/src/context/Context.jsx

```
import { createContext } from 'react';
export const UserContext =  createContext();
```

How Context API works?

Steps to create and use react context.

Provide Context

Wrap components needing access with a **<MyContext.Provider>**, passing the data via the **value** prop.

`/src/context/providers/UserProvider.jsx`

```
import { useState } from 'react';
import { UserContext } from "../Context";
const UserProvider = ({ children }) => {
        const [userContextValue, setUserContextValue] = useState({
                userid: "INDI001",
                username: "Rakesh",
                birthdate: "6th Jan, 2005",
        });
        return (
            <UserContext.Provider value={{ userContextValue, setUserContextValue }}>
                {children}
            </UserContext.Provider>
        );
};
export default UserProvider;
```

How Context API works?

Steps to create and use react context.

Wrapping Context

To get data globally we need to wrap provider with **App** or any component in which we want to operate the values.

/src/App.jsx

```
import router from "./routes/DataModeAllRoutes";
import { RouterProvider } from "react-router-dom";
import UserProvider from "./context/providers/UserProvider";

const App = () => {
    return (
        <UserProvider>
            <RouterProvider router={router} />
        </UserProvider>
    );
};
export default App;
```

How Context API works?

Steps to create and use react context.

Consume or Use Context

The way we have used **useContext** for **ContextTest.jsx,** the same way we can use for any component which can be located on anywhere in the project either it can be nested component or layout component.

Use the **useContext** hook in functional components within the Provider's tree, passing the context object to access the value from the nearest Provider.

/src/pages/ContextTest.jsx

```
import { useContext } from 'react';
import { UserContext } from "../../context/Context";
const ContextTest = () => {
    const { userContextValue, setUserContextValue } = useContext(UserContext);
    const handleOnClick = () => {
        setUserContextValue((prev) => ({
            ...prev,
            username: "Indixpert",
        }));
    };
    return (<div>
            This is default context / global username : {userContextValue.username}<br/>
            <button onClick={handleOnClick}>Click to change username</button>
        </div>
    );
};
export default ContextTest;
```

How Context API works?

Let's create a Product Wishlist example using **useReducer** hook

For Products JSON data you can directly copy from this url for demo purpose
Products JSON

What is Product Wishlist ?

A product wishlist is a feature in e-commerce that allows users to save items they are interested in for later purchase without adding them to their cart immediately. It's essentially a personalized list of products a customer wants to keep track of or potentially buy in the future. Wishlists can be used for various purposes, including gift ideas, reminders, or simply saving items for personal consideration.

Create Context

Use **createContext**() to create a context object for shared data.

/src/context/Context.jsx

```
import { createContext } from 'react';
export const WishlistContext =  createContext();
```

How Context API works?

Create Provider Context

Wrap components needing access with a **<MyContext.Provider>**, passing the data via the **value** prop.

`/src/context/providers/WishlistProvider.jsx`

```
import { useReducer } from 'react';
import { WishlistContext } from "../Context";
import { WishlistReducer } from "../reducers/WishlistReducer";

const WishlistProvider = ({ children }) => {
        const { wishlistState, wishlistDispatch } = useReducer(WishlistReducer, {
                userId : 1,
                wishlistItems: [ ],
         });
        return (
            <WishlistContext.Provider value={{ wishlistState, wishlistDispatch }}>
                  {children}
            </WishlistContext.Provider>
        );
};
export default WishlistProvider;
```

How Context API works?

Create Reducer Functional Component

Here we will create the reducer functional component, in the same way we had created reducer for TodoList.

/src/context/reducers/WishlistReducer.jsx

```
import { v4 as uuid } from "uuid";
export const WishlistReducer = (state, action) => {
        const { type, payload } = action;
        switch (type) {
              case "ADD_TO_WISHLIST":
                    return {
                          ...state,
                          wishlistItems: [
                                ...state.wishlistItems,
                                { id: uuid(), productID: payload.id, title: payload.title,
                                 price: payload.price, thumbnail: payload.thumbnail, },
                              ],
                          };
              case "REMOVE_FROM_WISHLIST":
                    return {
                          ...state,
                          wishlistItems: state.wishlistItems.filter((product) => product.id !== payload),
                    };
              default:
                    return state;
 } };
```

How Context API works?

Method 1

Wrapping Nested Context

To get data globally we need to wrap provider with **App** or any component in which we want to operate the values. Here if we have already any other **Provider**, we can wrap it within that provider to get data of both provider together.

/src/App.jsx

```
import router from "./routes/DataModeAllRoutes";
import { RouterProvider } from "react-router-dom";
import UserProvider from "./context/providers/UserProvider";
import WishlistProvider from "./context/providers/WishlistProvider";
const App = () => {
        return (
            <UserProvider>
                <WishlistProvider>
                  <RouterProvider router={router} />
                </WishlistProvider>
            </UserProvider>
        );
};
export default App;
```

How Context API works?

Method 2

Wrapping Context using ProviderComposer

Wrapping RouterProvider with Provider Composer.

/src/App.jsx

```
import router from "./routes/DataModeAllRoutes";
import { RouterProvider } from "react-router-dom";
import UserProvider from "./context/providers/UserProvider";
import WishlistProvider from "./context/providers/WishlistProvider";
import ProviderComposer from "./utils/ProviderComposer";
const App = () => {
    const providers = [UserProvider, WishlistProvider];
    return (
        <ProviderComposer providers={providers}>
            <RouterProvider router={router} />
        </ProviderComposer>
    );
};
export default App;
```

/src/utils/ProviderComposer.jsx

```
const ProviderComposer = ({ providers, children }
) =>  providers.reduceRight(
   (acc, Provider) => <Provider>{acc}</Provider>,
   children
 );
export default ProviderComposer;
```

How Context API works?

/src/pages/Products.jsx

The code shown here for the product listing page is partial code, which depicts the concept of add to wish list button functionality.

```
// Required imports need to place here....
const Products = () => {
    const {  wishlistState, wishlistDispatch } = useContext(WishlistContext);
    const AddToWishlistButton = ({ product }) => {
        const found = wishlistState.wishlistItems.filter( (item) => item.id === product.id )?.length > 0 ? true : false;
        const handleClick = (product) => {
            if (!found) { wishlistDispatch({ type: "ADD_TO_WISHLIST", payload: product,}); }
        };
        return (
            <Button size="sm" variant="outline-success" onClick={() => handleClick(product)}>
                {found ? <Check size={20} /> : ""} Add To Wishlist
            </Button>
        );
    };
    return ( <>
            {ProductsData.map((product) => {
                return (
                    ..........
                    <AddToWishlistButton product={product} />
                    ..........
                ); })}
        </> ); };
export default Products;
```

Exercise

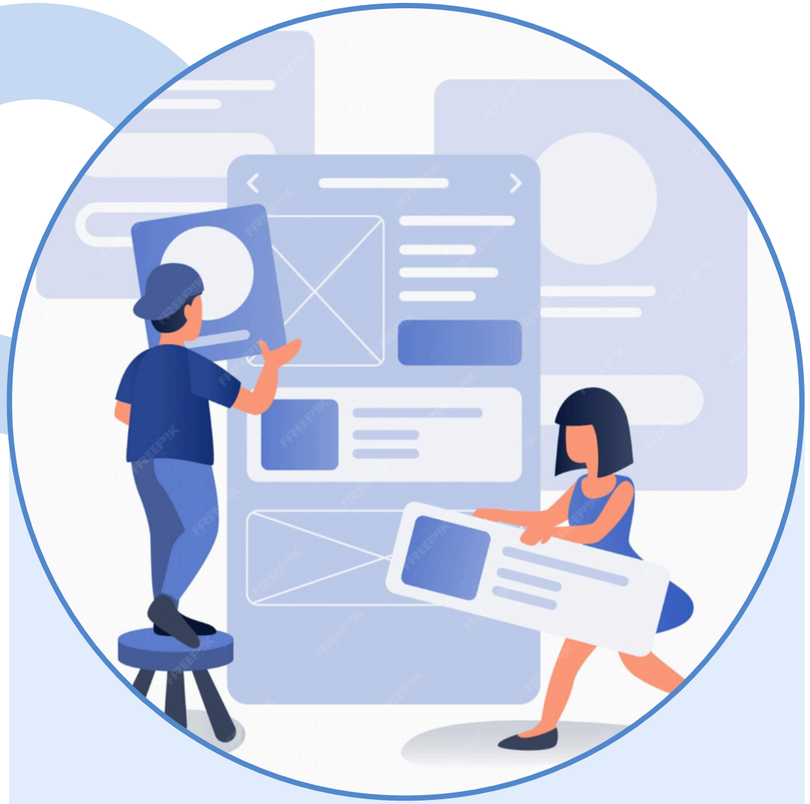
Create new section in react-bootstrap project named **Context** and implement the wishlist and addtocart functionalities as described below, using **react Context API**.

- Products Listing with Add To Wishlist and Add to Cart buttons, which contains details like Product image, name, add to wishlist button, quantity box and add to cart button.
- On clicking Add to wishlist, item should be added to wishlist, if item is already added, show notification it's already added else show notification "Item successfully added in wishlist", also add facility to remove item from wishlist and move it to cart using Add to Cart button.
- On clicking Add to cart, item should be added to Cart, if item is already added, update specified quantities, also show notification for various actions like add, update or delete item from cart.

For Product and categories data you can use below JSON data
https://dummyjson.com/products

Conclusion

We have learn how to manage global state and based on that we are now able to develop application like eCommerce, having a functionalities like add to cart, add to wishlist, compare products etc...



THANK YOU!

indiXpert